

NDNSF: A DATA-CENTRIC SERVICE FRAMEWORK FOR SECURE, COLLABORATIVE,
AND DYNAMIC NETWORKED APPLICATIONS

by

Tianxing Ma

A Dissertation Proposal

Submitted for Committee Review

Academic Advisor:

Dr. Lan Wang

Major: Computer Science

The University of Memphis

September 2026

Abstract

This proposal studies how an NDN service framework can discover currently willing Providers, authorize execution, and enforce task-specific named-data dependencies. NDNSF uses ABE-backed authorization to protect discovery descriptor Content from parties lacking the required decryption capability, while reusing identity-signing credentials independently of service entitlements. Authorized but unselected Providers can still read that descriptor; selected input and result exchanges use request-scoped protection.

The proposed collaboration extension constructs role assignments from eligible positive ACKs and derives Provider-local checks for named dependencies. These checks add task-specific constraints beyond existing request-ID isolation; neither signatures nor a signed plan alone establish honest results or globally unique commitment. UAV multi-view collection and distributed inference serve as validation workloads for the same research questions.

Existing campaigns provide bounded evidence on invocation, mobility switching, execution starts, and a small-model inference diagnostic. They do not qualify all current collaboration or native inference features. The remaining work compares complete authorization alternatives, tests incorrect role/dependency and recovery cases, and measures overhead and conditions with no benefit. The proposal separates implementation, local tests, measured findings, and open acceptance gates, with a conditional Spring 2027 completion plan.

Keywords: Named Data Networking, service authorization, provider discovery, collaboration, distributed inference

Table of Contents

Chapter	Page
1 Introduction	1
1.1 A Task That Requires Runtime Decisions	1
1.2 What NDN Already Provides	1
1.3 Thesis and Research Questions	2
1.4 Contribution and Evidence Boundary	3
2 Related Work and Design Alternatives	4
2.1 Invocation Is Not the Unsolved Problem by Itself	4
2.2 Three Complete Authorization Designs	4
2.3 From a Confidentiality Requirement to an Architectural Benefit	6
3 NDN System Model and Protocol	7
3.1 Participants, Trust, and Exclusions	7
3.2 Four Named Message Types	9
3.3 Authorization and Confidentiality	9
3.3.1 Rationale for ABE-Backed Service Authorization	9
3.3.2 Authorization Mechanisms, Lifecycle, and Costs	11
3.4 From ACKs to a Collaboration Plan	13
3.5 Object Admission and Execution State	13
3.6 Conditional Correctness Argument	14
4 Application Validation	16
4.1 UAV: Multi-View Collection, Processing, and Return	16
4.2 Distributed Inference: Explicit Data and State Dependencies	17
4.3 Current Scope and Completion Gates	18
5 Available Evidence and Required Evaluation	19
5.1 Measured Findings Are Configuration-Specific	19
5.2 Evidence Needed for Each Research Question	20
6 Research Plan, Risks, and Decision Gates	22
6.1 Scope Before More Features	22
6.2 Limits That Must Remain Visible	23
7 Conclusion	24
References	24

Chapter 1

Introduction

1.1 A Task That Requires Runtime Decisions

Consider an operator requesting several views of a ground location. Nearby UAVs may have different viewing directions, sensors, battery levels, and availability. A compute-capable UAV or edge node can analyze the collected images. The operator knows the required services but cannot assume which Providers will be reachable and willing to participate when the request is issued. The application must discover candidates, assign distinct collection and processing roles, and associate each intermediate image with its assigned producer and consumer. This is the motivating workload, not a claim that multi-view recognition itself is new.

A discovery descriptor contains the information needed to decide whether to offer service: for example, a sufficiently precise area, deadline, sensor requirement, and processing requirement. Application input contains the material needed to execute the selected task. The boundary is semantic, not a fixed field list: a location needed for feasibility belongs in the descriptor, whereas an image needed only for processing can be provided after Selection. Disclosure of an area and deadline can reveal an operator's intended activity even without revealing the images. We therefore study discovery that keeps descriptor Content confidential from parties lacking the required decryption capability. Authorized candidates, including those not selected, can read it; names and traffic patterns are not thereby hidden.

1.2 What NDN Already Provides

Named Data Networking (NDN) provides Interest-based retrieval of named Data, with signatures that support integrity and producer authentication under a trust policy [1, 2]. Caching can satisfy a later Interest without contacting the original producer; it does not guarantee persistent storage or delivery before a deadline. A signature alone does not establish execution permission, freshness for a particular task, or semantic correctness of a result. NDN Sync

announces changes in distributed datasets so applications can retrieve newly available Data [3,4]; it is not an unsolicited Data-push primitive.

NDN service and computation frameworks already exist, including NFN, NFaaS, RICE, and NSC [5–8]. NDNSF is not motivated by their absence. The research concerns the integration and evaluation of runtime candidate discovery, service authorization, and task-specific dependency enforcement. A broad claim that host-oriented systems cannot support these functions would also be incorrect: for example, gRPC supports extensible name resolution [9]. Comparisons must examine complete configurations and their costs.

1.3 Thesis and Research Questions

The thesis to be evaluated is that a service framework can use named, verifiable objects to connect runtime Provider selection to explicit execution rules, while separating service-level discovery confidentiality from request-specific input and result confidentiality. The expected benefit is conditional: additional discovery, validation, and key-management work must be justified by the task’s mobility, authorization, or collaboration requirements.

RQ1. How can a requester discover and authorize currently willing Providers without making the discovery descriptor public, and what are the costs relative to complete alternative authorization designs?

RQ2. How can Providers selected at runtime execute a task according to its roles and named dependencies, whether intermediate Data is retrieved from producers or caches?

RQ3. Under what workload and network conditions do these mechanisms improve completion, latency, or execution efficiency, and when do their costs outweigh their benefits?

UAV sensing and distributed inference are complementary validation workloads for these questions, not separate research contributions simply because two applications are implemented. Recovery is part of the protocol and evaluation scope; it is not assumed to be an independent novel contribution.

1.4 Contribution and Evidence Boundary

The intended contributions are (1) a justified and evaluated discovery/authorization design, (2) a protocol that converts runtime role assignments into enforceable named-object admission rules, and (3) bounded empirical findings about their benefits and costs. ABE, Trust Schema, Sync, caching, model partitioning, and challenge-response are reused mechanisms, not newly invented primitives. The work claimed here concerns their NDN-SF-specific design and evaluation. The final dissertation must identify the candidate's implementation, analysis, and experiments separately from collaborators' work and reused software; this proposal does not assign unverified personal ownership to shared code.

This is a revised proposal, not a completed dissertation. The submitted v0.1 implementation, subsequent measured campaigns, current implementation work, and proposed acceptance conditions are separate evidence categories. Existing request IDs already provide request-level binding; a hypothetical stale recovery attempt is not a defect established in a single-attempt historical protocol. Current collaboration and native distributed-inference work must pass its own end-to-end gates. Chapter 5 reports only the limited findings presently available.

Chapter 2

Related Work and Design Alternatives

2.1 Invocation Is Not the Unsolved Problem by Itself

NFN studies named computation and expression evaluation; NFaaS studies execution placement in a named-function environment [5, 6]. RICE provides an invocation handshake with support for authentication and security information [7]. NSC specifies signed invocation Interests validated using an application-provided Trust Schema, and discusses protected input and result exchange [8]. These are positive precedents for invocation and authorization, not insecure straw-man baselines. A workload-specific implementation of NSC used in an experiment is not the entire NSC design.

For NDNSE, the narrower question is how a service request gathers current willingness and feasibility information before selecting execution roles, and how the resulting assignment constrains intermediate-object use. Long-term permission to offer a service does not by itself determine which authorized Provider is assigned to a particular predecessor role. Neither a request ID nor a valid signature alone expresses that assignment. Conversely, role and edge checks are not automatically novel merely because their fields are placed in a manifest. A final novelty claim requires a feature-level comparison with service composition, workflow security, and capability-based systems, including CaSCON [10], rather than a claim of being the first NDN service framework.

2.2 Three Complete Authorization Designs

DNMP is a concrete example of role-constrained command signing and verification [11]. A role can cover multiple actions; different permission sets do not necessarily require one new role or certificate per service. The per-service certificate alternative in this proposal is our design choice, not a limitation inherent to every Trust Schema. In that alternative, a User uses a caller-role certificate for Request and Selection; a Provider uses a provider-role

Table 1: Alternatives under the same discovery and execution requirements.

Design	Execution decision	Non-public discovery	Management cost
DNMP-inspired roles	Validate signed commands against role and command rules	Add encryption and a group/key-distribution design; signatures alone do not conceal Content	Role credentials, trust rules, status or expiry checks, and encryption keys
Per-service role certificates	Validate message type, service role, identity, and request state	Add the same confidentiality functionality before comparing costs	Potentially multiple service-role credentials per identity, plus encryption and revocation state
NDNSF ABE-backed	Validate identity signatures, encrypted challenges, and request/authorization state	Encrypt a descriptor for a service-authorized receiver set without listing current candidates	Identity credentials, ABE policies/DKEYs, status distribution, and rekeying

certificate for ACK and Response. Validators check the issuer chain, service and message-type rules, validity/status, request identity, and message lifecycle before accepting an action.

Recipient-specific encryption protects selected exchanges. Protected discovery additionally needs a group-distribution or attribute-based encryption mechanism. Trust Schema alone is therefore insufficient for confidentiality, but a complete schema-based system can satisfy it with encryption.

For revocation, a credential-based alternative must reject a revoked credential using sufficiently current status information or wait for expiry. Removing a certificate from a repository cannot invalidate copies already held by a validator. This is a proposed lifecycle requirement for a fair alternative, not a claim that the original DNMP paper implements NDNSF’s runtime revocation protocol.

2.3 From a Confidentiality Requirement to an Architectural Benefit

NDNSF's two connected reasons are confidential discovery before Provider selection and separation of dissemination from fine-grained content access. The first protects the descriptor needed for a participation decision. The second allows shared delivery of ciphertext while individual policy keys preserve different access scopes. NAC-ABE supplies attribute-based control of Data decryption; the enhanced library includes KP-ABE support [12]. Service-permission aggregation and identity-credential reuse support this architecture. Challenges connect decryption capability to invocation checks; signatures and request state remain necessary. Withdrawal and rekeying belong to the authorization lifecycle, not a third superiority claim. Section 3.3.1 develops the two reasons and their limits.

The comparison must include group-key or broadcast-encryption alternatives where appropriate. A stable, small group can make conventional group encryption attractive; known Providers and simple permissions can favor signed invocation with recipient encryption. ABE becomes attractive when recipients are identified by service entitlement before their current identities are known, and multiple entitlements should remain independent of identity-signing credentials. Policy size, membership churn, encryption cost, and authority availability bound this argument.

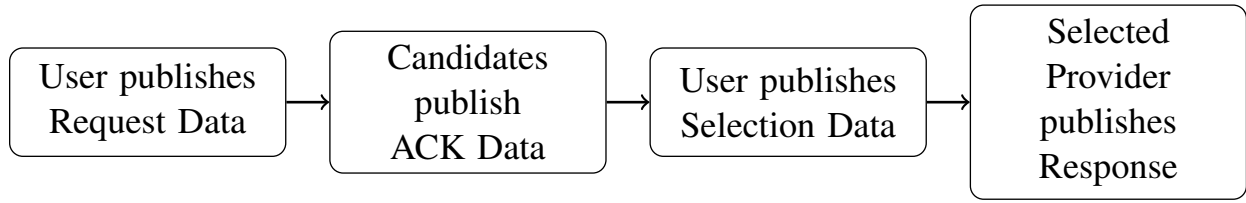
Chapter 3

NDNSF System Model and Protocol

3.1 Participants, Trust, and Exclusions

These are protocol roles, not necessarily separate devices: one node may be both a User and a Provider. The managed organization registers identities and approves service permissions; the Controller administers that policy. Participants require a provisioned trust anchor and validation rules for identity certificates and Controller material. Not knowing which candidates will respond to this invocation does not mean that the authorization authority has never enrolled those entities. Billing, arbitrary business transactions, globally optimal scheduling, and protection against all denial-of-service traffic are outside the claimed scope. The User requests and coordinates an invocation. Providers offer and execute services. A Controller administers authorization material and signed status. Forwarders and caches can deliver Data without being trusted to authorize execution. The model trusts the Controller and the User's planning decision; it does not claim consensus against a malicious coordinator. An attacker may observe visible names and traffic, inject invalid objects, replay previously valid messages, or operate without the needed service permission. Credential compromise and intentional sharing by an authorized principal require additional controls and are not prevented by ABE alone.

Authorization is permission to perform an operation, authentication establishes the signing identity under trust rules, and confidentiality limits who can recover Content. Valid credentials do not prove honest load reports, correct inference, or truthful sensor observations. Signed status must be obtained and validated before it can affect local authorization; a disconnected participant cannot be assumed to learn revocation immediately. Deadline/expiry checks require an explicit clock policy. Tests must report the allowed skew and failure behavior rather than silently assume synchronized clocks.



Controller: authorization material and signed status.
 Sync: discover names. Interests: retrieve Data.
 Validators: check accepted objects and local state.

Figure 1: Logical invocation dependencies. Arrows express application causality, not unsolicited network Data delivery.

Table 2: Logical message responsibilities; this is not a wire-format listing.

Data	Purpose	Acceptance condition
Request	Publish service requirements, minimized discovery descriptor, request identity, and Requester Challenge	Validate signature and current authorization context; recover the protected descriptor before evaluating willingness
ACK	Report positive or negative participation decision; a positive ACK carries selection information and Provider Challenge	Validate sender and request binding; only eligible positive ACKs enter planning
Selection	Authorize the chosen Provider/role and supply protected input or input references and request-scoped key material	Check intended Provider, challenge, request state, authorization status, and applicable plan rules before execution
Response	Publish completion/result metadata or references to protected results	Check authorized producer, request/plan context, result identity, and decryption scope

3.2 Four Named Message Types

These are NDNFSF Data objects. Sync supports discovery of their names, after which Interests retrieve Data, possibly from a cache. A negative ACK is an application participation decision, not a network Nack. Not every observed Request must receive a reply: invalid, unauthorized, duplicate, or expired messages are subject to validation and state checks. A positive ACK expresses willingness; it does not itself start the selected task or guarantee later resource availability.

3.3 Authorization and Confidentiality

3.3.1 *Rationale for ABE-Backed Service Authorization*

NDNSF's rationale has two connected parts: a security requirement for confidential discovery before Provider selection, and an architectural benefit of separating dissemination from fine-grained access to protected content. The first explains what must remain confidential; the second explains how different access scopes can coexist with shared dissemination. These are not two independent proofs of ABE's superiority. Permission aggregation and identity-credential reuse support the second part; permission withdrawal is a lifecycle requirement with costs, not a third reason to prefer ABE. Execution authorization determines whether an entity may invoke or provide a service; content confidentiality determines who can read protected message content. Signature validation, service permission checks, and invocation-state checks remain distinct requirements.

Reason 1: confidential discovery before Provider selection. Before Provider selection, the User publishes a discovery descriptor containing the location, deadline, and resource requirements needed for a Provider's positive-ACK decision. For example, nearby UAVs need an observation area and arrival deadline to decide whether to participate, but do not yet need the full application input. ABE protects this descriptor for Providers authorized for the service without enumerating their identities. Confidentiality is against parties lacking the required decryption

capability: authorized but unselected Providers can still read the descriptor. Content encryption does not hide outer Data names or traffic patterns. In the request-scoped confidentiality path, the full application input and its decryption material are restricted to selected Providers through Selection. A Trust Schema alone does not encrypt content; a role-authorized design can also use ABE or service-group encryption to provide discovery confidentiality.

Reason 2: separating dissemination from fine-grained access control. The set of nodes that can retrieve encrypted Request Data need not be the set that can read its discovery descriptor. Forwarders, caches, and repositories can carry or retain ciphertext without receiving its decryption keys. The User can publish according to service attributes rather than construct a recipient identity list for each Request; the Controller still authorizes entities and issues their keys. This separates delivery from disclosure, rather than delegating plaintext filtering to each distribution node.

Different entities can hold overlapping service entitlements while using the same dissemination mechanism. A UAV application may, for example, authorize one User for observation and detection services and another for detection and mapping services. This is an illustrative service-permission distribution, not evidence that arbitrary task, location, or time predicates are implemented. Fine-grained access survives aggregation only when the relevant objects retain their separate cryptographic scopes: encrypting a combined plaintext under one condition exposes the entire plaintext to anyone satisfying that condition.

In KP-ABE, ciphertext is associated with attributes and a decryption key (DKEY) encodes an access policy [12]. NDN-SF associates Provider access with `/SERVICE/<service>` and User access with `/PERMISSION/<service>`. Within one authority and ABE parameter generation, it combines an entity's service entitlements into an OR policy for its individually issued DKEY, while reusing its identity-signing credentials. Trust Schema specifies acceptable signing relationships; by itself it does not restrict who can read copied Content [13]. This distinction applies beyond role-based designs. Trust Schema can

express fine-grained rules and can be combined with encryption. NAC also protects distributed content through encryption and key distribution [14]; ABE’s distinctive contribution here is attribute-policy-based decryption, not an exclusive ability to protect cached data or reuse credentials.

DNMP constrains command targets and actions through role signing keys and a Trust Schema [11]. Existing roles may cover several permissions; different entitlements may require changes to role assignments, credentials, or rules, not necessarily one new role per service.

For comparison, our per-service role-certificate alternative would issue each entity a certificate for each authorized service role. A User would use its service-role key to sign Request and Selection Data; a Provider would use its service-role key to sign ACK and Response Data. Validators would check the service, message type, certified role, certificate status, and invocation state; encryption would be supplied separately. ABE avoids these additional service-specific signing credentials, but still requires DKEY issuance and updates, with size and processing costs that can grow with policy complexity. Certificate-based designs can also reuse credentials; fewer credential objects alone do not establish lower total cost.

3.3.2 Authorization Mechanisms, Lifecycle, and Costs

Connecting decryption capability to invocation authorization. The Requester Challenge (`UserToken` in the implementation) and Provider Challenge (`ProviderToken`) check access to the required service and permission decryption capabilities in the current exchange. Signatures identify the participants, while matching and one-time-state checks reject mismatched or replayed messages. This reuses attribute-based permission management instead of adding service-role signing credentials. It is not cost-free: confidential discovery already needs Provider DKEYs, whereas User permission DKEYs and challenge processing are additional authorization costs. ABE confidentiality combined with role-certificate authorization remains a valid alternative.

Runtime permission withdrawal and evaluation. The current runtime implements permission withdrawal through Controller-signed authorization status and checks at invocation processing points. A participant must retrieve and validate updated status before enforcing the newly withdrawn authority. ABE parameter rotation and refreshed DKEYs protect subsequent exchanges encrypted under the new generation. The present rotation is Controller-wide, so other authorized members may also need refreshed DKEYs; it is not a zero-fan-out revocation scheme.

DNMP describes encryption-key rotation to restrict subsequent access to measurement data, but does not detail a comparable online command-authorization revocation protocol [11]. A role-certificate comparison design can revoke the relevant credential and require execution nodes to check signed revocation status or reject expired certificates. This invalidates authority at the validator; it does not physically retrieve distributed certificates. Both approaches require update propagation, and neither retracts previously disclosed keys or plaintext.

Focused integration tests exercise NDNFSF's runtime revocation checks, including identity-, certificate-, and service-scoped withdrawal. These tests establish behavior at the exercised runtime boundaries, not network-wide instantaneous revocation or lower cost than certificate-based alternatives. The remaining evaluation should measure status-propagation delay, rejection delay, rekeying traffic, and effects on unaffected members. Runtime status validation and key rotation therefore extend NDNFSF to permission withdrawal, with explicit propagation and rekeying costs.

Discovery confidentiality applies to descriptor Content against parties lacking the required ABE decryption capability. It does not conceal the descriptor from other authorized candidates or provide traffic-analysis resistance. After Selection, the design uses request-scoped symmetric keys for application input and results; the selected Provider receives the necessary key material protected for its authenticated public key. Encrypting an image symmetrically and delivering its key in Selection is distinct from encrypting the image directly with a public key. The response key is shared by the requesting User and the selected Provider: other Users are excluded under the key-secrecy assumptions, but the Provider necessarily knows the plaintext it produces. This

paragraph specifies the intended protected exchange, not retroactive behavior of every v0.1 path.

3.4 From ACKs to a Collaboration Plan

The application supplies a role/dependency template, required services, feasibility constraints, and deadline. The template may be written manually; automatic generation is not necessary to establish the framework contribution. A replaceable planning policy selects among eligible positive ACKs after the collection window closes. Closing the window freezes the candidates observed for that decision, not all possible Providers in the network. ACK claims can become stale, so a selected Provider still performs execution-time admission and may reject work it can no longer execute.

The proposed signed Plan Manifest identifies the request, the accepted candidate snapshot, exact role-to-Provider assignments, permitted named dependency edges, authorization/key scope, and execution identity. Its digest is computed over a canonical plan body that excludes the digest field itself. Provider-local rules are derived from this plan; they are not inferred merely from a producer's long-term certificate. One User-side logical commitment per execution identity prevents accidental conflicting local commits under the trusted-User model. A signature alone does not prove globally unique commitment or protect against a Byzantine User signing two plans.

3.5 Object Admission and Execution State

A downstream Provider accepts a dependency only if its Data signature validates under the applicable trust rules, its request/plan identity matches the active assignment, the signer is assigned to the predecessor role, and its name, schema, and key scope match a declared edge. Required segments must validate, and incomplete or malformed objects cannot satisfy an edge. The same checks apply to Data fetched from a producer, cache, or repository. Names belong to the actual producer's authorized namespace; the plan references those names rather than relabeling another producer's object as the consumer's own Data.

Table 3: Proposed task-state contract and required negative checks.

State/event	Permitted transition	Required rejection or bounded behavior
Collecting ACKs	Close window and validate candidate set	Late ACK does not silently alter a committed plan
Selected	Validate Selection and perform local admission	Positive ACK alone, wrong Provider, or spent challenge cannot start work
Waiting for dependencies	Execute when required authorized edges are satisfied	Wrong role, undeclared edge, incomplete object, or wrong active execution is rejected
Completed/cancelled	Return recorded status under duplicate-message policy	Duplicate publication does not implicitly repeat a side effect
Failure/reassignment	Explicitly terminate or authorize a replacement execution	Old execution must not acquire authority from the new assignment; allowed result reuse is explicit

These are proposed acceptance conditions, not a claim that every production path is qualified. Network re-retrieval is not identical to repeating computation. If a task is reassigned, cancellation, fencing, and result-reuse rules must be explicit. An already computed image may remain useful when the new plan explicitly authorizes its reuse; an old object is not automatically invalid solely because it is cached. Application-defined compensation can limit side effects but does not provide distributed atomic transactions or exactly-once physical action.

RQ1 is addressed by the authorization design and its qualified tradeoffs. RQ2 is addressed by the proposed plan/state contract. Existing invocation evidence does not establish collaboration correctness; wrong-role, wrong-edge, replay, cancellation, and direct/cache tests remain separate acceptance obligations.

3.6 Conditional Correctness Argument

Assume uncompromised signing keys, correctly enforced trust/authorization checks, and Providers that obey the specified state machine. Selection-before-execution follows only if every

handler-start path checks an active assignment and unspent authorization state. Dependency integrity follows only if every input path, including a cache hit, checks the assigned producer and declared edge before satisfying the dependency. Replacement isolation additionally requires old workers and late callbacks to lose authority under the new execution identity. Restart recovery must preserve or safely invalidate the relevant one-time state; signature validity and cache FreshnessPeriod alone are insufficient.

These are conditional invariants and implementation proof obligations, not a cryptographic proof of a complete system. Tests should inject a wrong-role object within the same request, an undeclared edge from a correctly signed producer, and a late callback after replacement. Each isolates a property that a request ID alone cannot express. Application result correctness and physical side effects remain outside this argument.

Chapter 4

Application Validation

4.1 UAV: Multi-View Collection, Processing, and Return

The primary UAV case is a ground-location request, not a complete autonomous mission system. An operator chooses a target location or area on a map. The Request reveals only the service-level-protected information needed to assess coverage, viewing feasibility, sensor capability, and timing. Eligible UAVs return positive ACKs; a planner assigns complementary viewing roles and a processing role. Selection delivers the execution details and appropriate keys to the chosen Providers.

Each sensing UAV publishes encrypted, signed ViewImage Data in its own namespace, with observation time and available pose/calibration metadata. GPS helps relate platforms and target areas, but does not by itself establish line of sight, camera direction, accurate depth, or correspondence between views. The application must define capture tolerances, view adequacy, and failure handling. Different views cannot be treated as identical cropped inputs without matching or geometric reasoning.

The compute Provider retrieves the authorized images by their named dependencies, performs detection or other specified analysis, and publishes signed annotation Data. Its result references the original image Data and securely supplies the requesting User with the permitted image keys. This reuses the original signed images; it does not republish them under a false producer identity. The User retrieves those images and annotations and renders the overlay. Improved recognition or localization accuracy is not assumed: such a claim requires ground truth, a single-view control, and a specified fusion method.

MiniNDN [15] tests named-data execution under controlled connectivity; PX4 SITL [16] can test the application/autopilot interaction. Neither is a real-flight safety certification. The framework-level endpoint is authorized multi-view collection, processing, and return before a deadline. Other flight-control, video, recording, and patrol features remain supporting

engineering, not independent dissertation completion conditions.

4.2 Distributed Inference: Explicit Data and State Dependencies

The second workload partitions a specified model across Providers and exchanges named intermediate objects. Pipeline partitioning assigns sequential layers or layer groups; tensor partitioning divides operations/tensors and requires the appropriate communication and collectives. The terms describe different execution structures, not interchangeable drawing orientations. ONNX [17] represents computation graphs, but export alone does not implement cross-Provider collectives or prove distributed numerical correctness. Model partitioning has substantial prior work [18, 19]; the proposed NDNSF increment is runtime assignment and checked named dependency execution, not partitioning itself.

In the proposed execution, the planner combines a supported model/partition description with ACK capabilities. Providers obtain authorized artifact references, prepare their assigned fragments, and exchange activations or collective inputs over the declared edges. A repository provides managed artifact storage; the forwarding Content Store is opportunistic caching, not a persistence guarantee. Reuse must distinguish an identical model artifact from request-specific activation or execution state.

For autoregressive generation, the model computes K/V tensors; the runtime retains and supplies the relevant cache state to later decoding steps. Keeping model weights resident does not eliminate this state dependency. Pipeline execution still has a temporal feedback loop: generated-token/state information is needed for the next step. If LLM streaming is retained in the final claim, evaluation must separate prefill, time to first token, per-token decoding, and KV storage/transfer. It is not necessary to qualify every model or hybrid partition merely to validate RQ2.

4.3 Current Scope and Completion Gates

Current native implementation work and local tests are evidence of engineering progress, not interchangeable with cross-process, multi-device, or sealed-container qualification. As of the September 2026 revision, the V3/native integration and compatibility gates are not all closed. The historical small-model campaign in Chapter 5 does not qualify canonical-layer reuse, tensor/hybrid execution, multi-accelerator placement, or recovery of a large language model.

The final campaign will freeze model version, precision, supported partition, device memory, network conditions, warm/cold state, and an output-comparison tolerance. It will include a single-Provider baseline when feasible. If the model cannot fit one device, distributed feasibility is meaningful, but no speedup ratio against a non-running baseline is claimed. UAV and DI jointly test the role/dependency abstraction; their task algorithms and device-specific adapters remain application responsibilities.

Chapter 5

Available Evidence and Required Evaluation

5.1 Measured Findings Are Configuration-Specific

The following numbers are retained from recorded campaigns, not new experiments in this revision [20]. The independent unit for comparative uncertainty is the seed/run, not each correlated request. Success-conditioned latency must be read with completion and timeout rates. A baseline label identifies the evaluated harness configuration, not every feature of the framework named in that label.

Table 4: Selected evidence and limits; values are unchanged from their records.

Campaign	Observation	Supported interpretation / limit
Ten-seed switching holdout	1,312 matched switching requests; NDNSF minus gRPC seed-p95 difference: -597 ms, 95% CI $[-992, -200]$; versus NSC: -2618 ms, CI $[-2916, -2315]$	Lower conditional tail latency in the tested four-Provider configuration, not universal mobility or success-rate superiority
Six-seed work-efficiency repeat	NDNSF: 1,798/1,800 completed, 1,800 execution starts; gRPC-PAR-4: 1,800/1,800, 7,180 starts	Selection avoids redundant starts relative to this parallel policy; starts are not messages, bytes, CPU time, or energy
Earlier small-model DI diagnostic	Ten successful runs/layout: mean 259.4 ms distributed versus 186.8 ms single Provider; p95 283.9 versus 206.3 ms	Splitting was slower for this model/configuration; small success-conditioned sample does not establish a general tail-latency result

In the work-efficiency comparison, NSC-4 also completed 1,800/1,800 requests, with 1.002 execution starts per request (rounded). This does not establish a work-efficiency advantage over NSC. The mobility campaign uses common traces, publication times, deadlines, and four Providers per system. The switching subset is defined by reachability and the registered classification, not chosen because NDNSF wins. Overall coverage controls and excluded/failed requests must remain alongside this conditional result in the final analysis. The reported small-model difference is consistent with communication/setup overhead, but no isolated

profiling result here establishes its cause.

The transition experiment reports 357/357 completions for NDNSF without pre-registering Provider addresses, 0/357 for static-list baselines, and 357/357 for pre-registered baseline variants. It isolates the discovery configuration. A gRPC resolver or other discovery integration can update a candidate list [9]; the result does not prove that gRPC or NSC cannot discover a new Provider.

Historical v0.1 loss results include 599/600 completions at 1% per-link loss, 598/600 at 3%, and a best recorded 539/600 at 10%. The last is a selected historical run, not a current aggregate estimate or a matched comparison against later baseline data. The expression $1 - (1 - \ell)^4$ assumes four independent packet traversals with equal loss ℓ ; it is not the full service-failure probability after retries. Timeouts under heavy loss do not, without further interventions, identify one exclusive cause or rule out protocol/policy effects.

5.2 Evidence Needed for Each Research Question

Table 5: Claim-driven evaluation plan, not a completed-results checklist.

RQ	Comparison	Correctness / failure observation	Costs and outcomes
RQ1	ABE-backed; role-schema plus equivalent encryption; suitable group-key alternative	Unauthorized call never starts handler; replay, wrong challenge, revoked/expired state, status delay	Completion, discovery delay, crypto work, credential/policy size, renewal and withdrawal traffic
RQ2	Same task graph with static/dynamic assignment; controlled removal of each admission check	Wrong role, edge, name, key or execution; direct/cache parity; incomplete objects; cancellation or reassignment	End-to-end completion, validation cost, intermediate bytes, redundant work, bounded recovery
RQ3	Matched Provider counts, tasks, capacities, deadlines and failure schedules; feasible single-device control	Include failed and timed-out requests; distinguish network, admission, execution and result-retrieval failures	Completion and latency together; cold/warm costs; per-run uncertainty; conditions with no benefit

Each campaign must preserve its source/dependency/model identity, configurations,

seeds, measurement start/end points, raw-run references, and analysis scripts. Scaling dimensions should follow the actual claim: candidates and services for discovery/authorization, edges and object sizes for collaboration. Ablation diagnoses a mechanism; an intentionally insecure variant is not a recommended deployment. An unqualified runtime start or collector failure is a harness failure, not a protocol failure rate.

Chapter 6

Research Plan, Risks, and Decision Gates

6.1 Scope Before More Features

The immediate priority is a frozen, runnable baseline for the final claims, not completion of every development feature. Select one UAV collection–processing graph and one supported DI partition first. Keep pipeline/tensor/hybrid generality, LLM streaming, and sophisticated recovery only where the implementation and evidence can support them. A negative performance result is an acceptable research finding when its conditions and costs are explained.

Table 6: Revised internal targets; defense is conditional on evidence and committee approval.

Period	Deliverable / decision gate
September 2026	Agree on three RQs and bounded claims; freeze supported application graphs and implementation/evidence baseline; resolve correctness blockers
October 2026	Execute registered correctness and matched comparison cases; reduce optional scope if an end-to-end gate remains open
November 2026	Freeze accepted results and provenance; write RQ answers, costs, and limitations; keep missing results visibly unverified
December 2026	Deliver an integrated complete draft to the advisor; no new optional feature branch in the research argument
January 2027	Advisor/committee review, revisions, defense readiness decision, and confirmation of current university submission/notice requirements
February 2027	Target defense only after required reviews and evidence closure; allow time for substantive corrections
Spring 2027	Complete approved corrections and final submission by the applicable Graduate School deadlines; confirm exact dates separately

The February 2027 defense is a target, not a statement of readiness or an institutional deadline. CS guidance calls for an advisor-approved complete dissertation to reach committee members two weeks before the defense [21]. The Graduate School currently lists March 26, 2027 for the defended final ProQuest submission in Spring 2027 [22]; confirm current instructions before filing. If core end-to-end evidence is still missing at the January review, the committee must reconsider scope or timing. Final-defense slides must replace planned tests with actual

answers; changing a label from “planned” to “completed” cannot satisfy that gate.

6.2 Limits That Must Remain Visible

ABE adds authority, computation, and rekeying costs; disconnected revocation has propagation limits. Candidate ACKs do not prove future capacity or honest reporting. Cache availability does not guarantee task completion. Local tests do not qualify distributed failure behavior. UAV safety and perception accuracy need evidence beyond service-message delivery. Hardware/model-specific findings cannot establish unlimited framework generality. These are conditions for interpreting the research, not reasons to omit an unfavorable result.

Chapter 7

Conclusion

NDNSF's proposed research contribution is a service-level protocol and evidence-backed design rationale, not a new encryption primitive or the first use of named computation. RQ1 asks when protected discovery and reuse of ABE capability are preferable to complete alternatives. RQ2 asks how runtime assignments become local rules for accepting named dependencies. RQ3 asks where the resulting costs buy useful completion, latency, or execution-efficiency benefits. Existing campaigns establish limited invocation findings; the collaboration and cross-application claims still require their declared end-to-end evidence. This separation provides a concrete route from the present proposal to a defensible dissertation.

References

- [1] L. Zhang, A. Afanasyev, J. Burke, V. Jacobson, kc claffy, P. Crowley, C. Papadopoulos, L. Wang, B. Zhang, Named data networking, in: ACM SIGCOMM Computer Communication Review, 2014, pp. 66–73.
- [2] Named Data Networking Project, NDN Packet Format Specification: Data Packet, <https://docs.named-data.net/NDN-packet-spec/current/data.html>, , accessed September 9, 2026.
- [3] T. Li, W. Shang, A. Afanasyev, L. Wang, L. Zhang, A brief introduction to ndn dataset synchronization (ndn sync), in: 2018 IEEE Military Communications Conference, 2018, pp. 612–618.
- [4] P. Moll, V. Patil, L. Wang, L. Zhang, Sok: The evolution of distributed dataset synchronization solutions in ndn, in: Proceedings of the 9th ACM Conference on Information-Centric Networking, 2022, pp. 33–44.
- [5] C. Tschudin, M. Sifalakis, Named functions and cached computations, in: 2014 IEEE 11th Consumer Communications and Networking Conference, 2014, pp. 851–857.
- [6] M. Krol, I. Psaras, Nfaas: Named function as a service, in: Proceedings of the 4th ACM Conference on Information-Centric Networking, 2017, pp. 134–144.
- [7] M. Krol, K. Habak, D. Oran, D. Kutscher, I. Psaras, Rice: Remote method invocation in icn, in: Proceedings of the 5th ACM Conference on Information-Centric Networking, 2018, pp. 1–11.

- [8] T. Meirovitch, L. Zhang, Nsc–named service calls, or a remote procedure call for ndn, in: NDN Technical Report NDN-0074, Revision 1, 2021, pp. 1–20.
- [9] gRPC Authors, Custom name resolution, <https://grpc.io/docs/guides/custom-name-resolution/>, , accessed September 9, 2026.
- [10] Rochester Institute of Technology, Barrios presents at ieee smartcomp, <https://www.rit.edu/news/barrios-presents-ieee-smartcomp>, , primary institutional record of CaSCON, SMARTCOMP 2025; cited for identification, not an exhaustive feature audit.
- [11] K. Nichols, Lessons learned building a secure network measurement framework using basic ndn, in: Proceedings of the 6th ACM Conference on Information-Centric Networking, 2019, pp. 112–122.
- [12] S. Dulal, T. Yu, S. Liu, A. R. Thieme, L. Zhang, L. Wang, Enhancing NAC-ABE to support access control for mHealth applications and beyond, arXiv:2311.07299 (2023). doi:10.48550/arXiv.2311.07299.
URL <https://arxiv.org/abs/2311.07299>
- [13] Named Data Networking Project, Schematizing and automating trust in named data networking, <https://named-data.net/publications/techreports/ndn-0030-2-trust-schema/> (2015).
- [14] Named Data Networking Project, NAC: Automating access control via named data, <https://named-data.net/publications/zhang2018nac/> (2018).
- [15] Minindn: A mininet-based ndn emulator, <https://github.com/named-data/mini-ndn>, , accessed 2026.
- [16] Px4 autopilot user guide, <https://docs.px4.io/>, , accessed 2026.
- [17] Open neural network exchange (onnx), <https://github.com/onnx/onnx>, , accessed 2026.
- [18] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, L. Tang, Neurosurgeon: Collaborative intelligence between the cloud and mobile edge, in: Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, 2017, pp. 615–629. doi:10.1145/3037697.3037698.
- [19] S. Teerapittayanon, B. McDanel, H. T. Kung, Distributed deep neural networks over the cloud, the edge and end devices, in: 2017 IEEE 37th International Conference on Distributed Computing Systems, 2017, pp. 328–339. doi:10.1109/ICDCS.2017.226.

- [20] NDNFSF Project, Proposal research revision: Claim and evidence audit, Companion repository document: docs/PAPER/proposal-defense/research-revision-audit.md, , september 9, 2026; includes frozen campaign paths, evidence limits, and unresolved research gates.
- [21] University of Memphis, Department of Computer Science, Information for computer science ph.d. students, https://www.memphis.edu/cs/pdfs/forms_grad_phd_programinfo.pdf, , accessed September 9, 2026.
- [22] University of Memphis Graduate School, Graduation information, https://www.memphis.edu/gradschool/current_students/graduation.php, , spring 2027 calendar, accessed September 9, 2026.